

CONTROLS SYSTEMS

Project 04

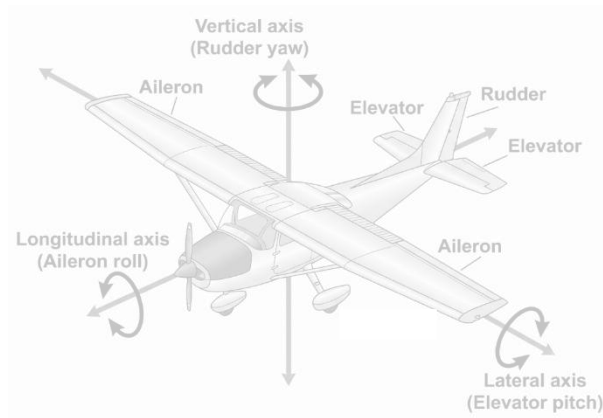
Aircraft Pitch Control System — Modeling, Analysis, and PID Control Using MATLAB and Simulink

Designed, Developed and Documented by

Zohaib Imtiaz

MATLAB R2024b • SIMULINK • Control System Toolbox • Aerospace Engineering

July 2026



Copyright © 2026 Zohaib Imtiaz. All Rights Reserved.

This report is the original work of the author and has been prepared for academic purposes. No part of this document may be reproduced, distributed, or transmitted in any form without proper acknowledgment of the author, except where permitted for educational or research purposes.

Abstract

This project presents the mathematical modeling, open-loop analysis, parameter sensitivity study, and closed-loop PID control of an aircraft longitudinal pitch dynamics system using MATLAB and Simulink. The aircraft pitch transfer function — a third-order system relating elevator deflection to pitch angle — was derived from linearized longitudinal equations of motion and implemented in MATLAB for comprehensive dynamic analysis. Open-loop pole analysis revealed two fundamental longitudinal flight modes: the Short Period mode (fast, heavily damped, pole at -1.2679) and the Phugoid mode (slow, lightly damped, complex poles at $-0.3336 \pm 0.1730j$). A right-half-plane (RHP) zero at $s = +1$ was identified, confirming the non-minimum phase nature of the plant and predicting the characteristic undershoot of 3.16% observed in the step response. A systematic parameter sensitivity study investigated the effects of static stability (a_0), dynamic stability (a_1), and pitch damping (a_2) on pole locations and transient performance. The study revealed that the system transitions between overdamped and underdamped regimes as parameters change, with dominant poles governing settling behavior. Closed-loop PID control was then implemented in Simulink with a reference pitch command of 5 degrees. P control alone revealed a critical gain margin between $K_p=0.4$ and $K_p=0.5$, beyond which the RHP zero causes instability — a fundamental limitation that does not occur in minimum-phase plants such as the DC motor in Project 03. PI control with $K_p=0.1$ and $K_i=0.01$ successfully eliminated steady-state error, reaching exactly 5 degrees, but with a settling time of 65 seconds due to the tight stability margin. MATLAB's automatic `pidentune()` algorithm produced $K_p=0.264$, $K_i=0.0372$, $K_d=0.455$, achieving a settling time of 19.5 seconds with 3.39% overshoot and 18.1% undershoot — confirming that the RHP zero signature cannot be eliminated by any PID controller. A wind gust disturbance of 2 degrees injected at $t=50s$ was fully rejected, with the aircraft returning to exactly 5 degrees commanded pitch by $t=83s$. The project conclusively demonstrates that PID control reaches its fundamental limits on non-minimum phase aerospace plants, directly motivating the state-space, pole placement, and LQR methods in Projects 07 through 09.

Table of Contents

Abstract	iii
List of Tables	vii
Table of Figures	viii
1 Objectives	1
2 Introduction.....	2
3 Aircraft Longitudinal Dynamics	3
3.1 Aircraft Axes and Rotational Degrees of Freedom.....	3
3.2 Why an Aircraft Needs a Pitch Controller?	4
3.3 Linearization and Why Transfer Functions Are Used	4
4 Mathematical Model	6
4.1 Transfer Function.....	6
4.2 The Right-Half-Plane Zero	7
5 MATLAB Open-Loop Analysis	8
5.1 Pole-Zero Analysis.....	8
5.1.1 Poles:.....	8
5.1.2 Zero:	8
5.1.3 Observation	9
5.2 Open-Loop Step Response.....	9
5.2.1 Response	9
5.2.2 Performance Specification	10
5.2.3 Observation	10
5.3 Sign Convention — Why K Was Flipped?.....	10
6 . Parameter Sensitivity Study	11
6.1 Effect of Static Stability (a_0)	11
6.1.1 Response	11

6.1.2	Poles and Performance Specifications	12
6.1.3	Observations:	12
6.2	Effect of Pitch Damping (a2).....	13
6.2.1	Response	13
6.2.2	Poles and Performance Specifications	14
6.2.3	Observations	14
6.3	Effect of Dynamic Stability (a1).....	15
6.3.1	Response	15
6.3.2	Poles and Performance Specifications	16
6.3.3	Observations:	16
6.4	Parameter Summary.....	17
7	Simulink Open-Loop Verification	18
7.1	Block Diagram	18
7.1.1	Response	18
7.1.2	Observations	19
8	. Closed-Loop PID Control	20
8.1	P Controller Analysis	20
8.1.1	Simulink Block Diagram	20
8.1.2	Responses — 30s and 100s Simulation	20
8.1.3	Performance Specifications	21
8.1.4	Gain Margin Discovery:	22
8.1.5	Observations	22
8.2	PI Controller Analysis.....	23
8.2.1	Simulink Block Diagram	23
8.2.2	Step Responses.....	23
8.2.3	Performance Specifications	24

8.3	PID Automatic Tuning — pidtune()	25
8.3.1	Response	25
8.3.3	Analysis of pidtune() Gains:	26
9	Controller Comparison.....	26
9.1.1	Observations	27
10	Disturbance Rejection — Wind Gust Test.....	28
10.1	Simulink Block Diagram	28
10.2	Disturbance Rejection Response.....	29
10.3	Observations:	30
11	Aerospace Relation	31
12	Conclusions.....	32
13	Future Work	34
14	Appendix.....	37
14.1	MATLAB Code	37
14.1.1	Open Loop Step Response	37
14.1.2	Closed Loop Step Response.....	40
14.1.3	Animation with Disturbance	41

List of Tables

<i>Table 1 : Aircraft Pitch Transfer Function Parameters</i>	<i>6</i>
<i>Table 2 : Open-Loop Poles and Associated Flight Modes</i>	<i>8</i>
<i>Table 3: Open-Loop Step Response Performance Specifications.....</i>	<i>10</i>
<i>Table 4: Effect of a_0 on Pole Locations and Performance.....</i>	<i>12</i>
<i>Table 5: Effect of a_2 on Pole Locations and Performance.....</i>	<i>14</i>
<i>Table 6: Effect of a_1 on Pole Locations and Performance.....</i>	<i>16</i>
<i>Table 7: Summary of Parameter Sensitivity Effects</i>	<i>17</i>
<i>Table 8: P Controller Performance — K_p Variation (100s simulation)</i>	<i>21</i>
<i>Table 9: PI Controller Performance — K_i Variation ($K_p=0.1$, 100s)</i>	<i>24</i>
<i>Table 10: pidtune() Automatic PID Gains and Performance.....</i>	<i>25</i>
<i>Table 11: Complete Controller Comparison — Aircraft Pitch System</i>	<i>26</i>
<i>Table 12: Wind Gust Disturbance Rejection Results.....</i>	<i>29</i>

Table of Figures

Figure 1 Aircraft Three-Axis Coordinate System — Pitch (θ), Roll (ϕ), and Yaw (ψ)	3
Figure 2 Pole-Zero Map — Aircraft Pitch Transfer Function	8
Figure 3 Aircraft Pitch Open-Loop Step Response 1 Degree Elevator Deflection (MATLAB)	9
Figure 4 Effect of Static Stability Coefficient (a_0) on Pitch Step Response	11
Figure 5 Effect of Pitch Damping Coefficient (a_2) on Pitch Step Response	13
Figure 6 Effect of Dynamic Stability Coefficient (a_1) on Pitch Step Response	15
Figure 7 Simulink Block Diagram — Open-Loop Aircraft Pitch Model	18
Figure 8 Open-Loop Step Response — Simulink Scope (Matches MATLAB)	18
Figure 9 Simulink Block Diagram — P Controller (Five K_p Values)	20
Figure 10 P Controller Step Responses for $K_p = 0.1, 0.2, 0.3, 0.4, 0.5$ (30s simulation)	20
Figure 11 P Controller Step Responses for $K_p = 0.1, 0.2, 0.3, 0.4, 0.5$ (100s simulation)	21
Figure 12 Simulink Block Diagram — PI Controller	23
Figure 13 PI Controller Step Responses — $K_i = 0.01, 0.03, 0.05, 0.07$ ($K_p=0.1, 100s$)	23
Figure 14 Automatic PID Tune Step Response	25
Figure 15 Simulink Block Diagram — Wind Gust Disturbance Rejection Test	28
Figure 16 Aircraft Pitch Response With and Without 2-Degree Wind Gust at $t=50s$	29

1 Objectives

The specific objectives are:

- Understand the physical meaning of aircraft pitch, roll, and yaw, and the role of the elevator in generating pitch moments.
- Study the linearized aircraft longitudinal transfer function and understand what each parameter represents physically.
- Identify the poles and zeros of the aircraft pitch transfer function and connect pole locations to the Short Period and Phugoid longitudinal flight modes.
- Understand and physically explain the right-half-plane (RHP) zero and its effect on step response as non-minimum phase behavior.
- Perform a complete parameter sensitivity study varying a_0 (static stability), a_1 (dynamic stability), and a_2 (pitch damping) individually.
- Verify the open-loop step response in Simulink and confirm it matches the MATLAB transfer function result.
- Design and analyze a P controller, identify the gain margin experimentally, and explain why P control alone cannot stabilize this plant with acceptable performance.
- Design and analyze a PI controller, demonstrate that integral action eliminates steady-state error, and identify the stability boundary for K_i .
- Use MATLAB's `pidtune()` to obtain automatic PID gains and compare against the fundamental performance limitations imposed by the RHP zero.
- Perform a wind gust disturbance rejection test and evaluate the autopilot's recovery capability.
- Establish why PID is insufficient for this plant and motivate advanced control methods for aerospace applications.

2 Introduction

Project 04 marks the point where control systems and aerospace engineering converge for the first time in this series. In Projects 01 through 03, the systems analyzed — a mass-spring-damper, a DC motor, and its PID speed controller — were mechanically simple and physically intuitive. The DC motor, while requiring careful PID tuning, was a well-behaved minimum-phase second-order plant with generous stability margins.

Aircraft pitch dynamics are fundamentally different. The longitudinal motion of a fixed-wing aircraft couples three physical phenomena simultaneously: forward translation through air (governed by drag and thrust), vertical heave motion (governed by lift and angle of attack), and rotational pitching about the center of gravity (governed by aerodynamic moments from the tail). This coupling produces a third-order system with characteristics that cannot appear in simple electromechanical plants.

The most important of these characteristics is the right-half-plane (RHP) zero — a direct consequence of how elevator deflection produces aerodynamic forces. When the pilot pulls the control stick back, the elevator deflects upward, which initially creates a downward force at the tail. This momentarily pushes the aircraft nose downward before the pitching moment rotates the nose upward. The aircraft initially moves in the wrong direction before correcting itself. This non-minimum phase behavior imposes a hard limit on how aggressively any controller can drive the pitch response, and it persists regardless of PID gain selection.

This project investigates these characteristics systematically. Each phase — open-loop analysis, parameter sensitivity study, and closed-loop PID design — builds directly on the previous projects while revealing fundamentally new control engineering concepts that motivate the modern control methods to follow.

3 Aircraft Longitudinal Dynamics

3.1 Aircraft Axes and Rotational Degrees of Freedom

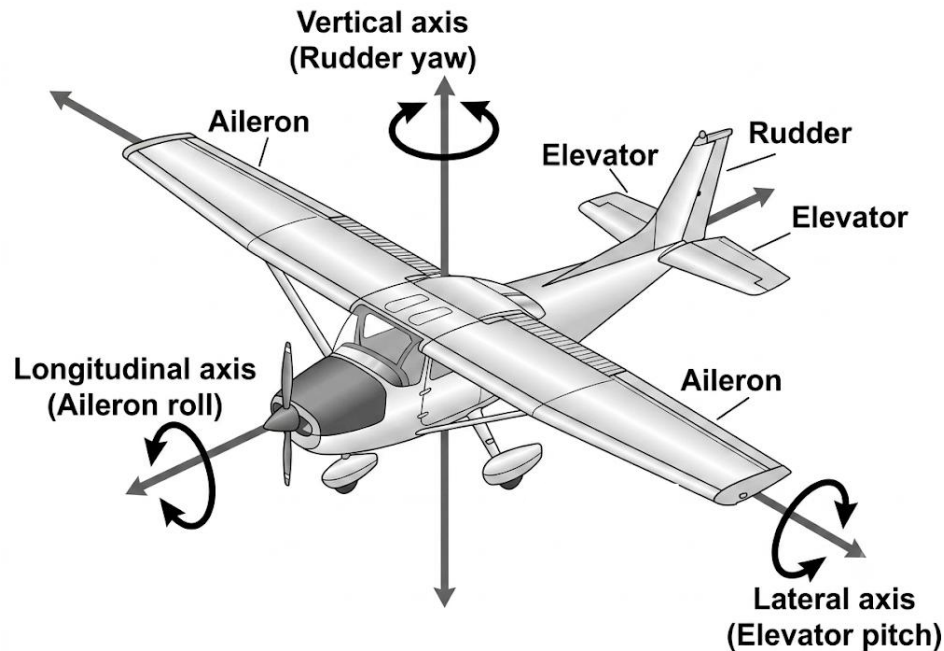


Figure 1 Aircraft Three-Axis Coordinate System — Pitch (θ), Roll (ϕ), and Yaw (ψ)

A rigid aircraft in flight has six degrees of freedom — three translational and three rotational. For this project, only the longitudinal motion is considered, which involves:

Pitch (θ): Rotation of the aircraft nose upward or downward about the lateral axis. Controlled by the elevator on the horizontal stabilizer. This is the output variable of the system.

The elevator is a movable control surface hinged at the rear of the horizontal stabilizer. *When the pilot pulls the stick back, the elevator deflects upward (δe positive), creating a downward aerodynamic force at the tail. This force generates a nose-up pitching moment about the aircraft center of gravity, rotating the aircraft to a higher pitch angle.*

3.2 Why an Aircraft Needs a Pitch Controller?

A pilot can manually maintain pitch angle, but three engineering requirements make an automatic controller essential:

- **Workload and Fatigue:** Maintaining precise pitch attitude over a long flight requires continuous stick input. Autopilot systems relieve this burden, allowing pilots to focus on navigation and decision-making.
- **Disturbance Rejection:** Atmospheric turbulence, wind gusts, and wake vortices continuously perturb aircraft pitch. An automatic controller can reject these disturbances far faster than a human pilot can perceive and respond to them.
- **Stability Augmentation:** Some aircraft configurations are naturally unstable in pitch and require active control to remain flyable. Modern high-performance fighters like the F-16 are deliberately designed to be statically unstable for maneuverability and rely entirely on automatic flight control systems to maintain controlled flight.

3.3 Linearization and Why Transfer Functions Are Used

The true equations of motion for an aircraft are nonlinear, coupling aerodynamic forces that depend on air speed, angle of attack, Mach number, and altitude in complex nonlinear ways. These equations cannot be analyzed using classical control tools such as poles, Bode plots, or root locus.

For small perturbations around a specific flight condition — a chosen airspeed, altitude, and trim attitude — the nonlinear equations can be linearized by Taylor expansion. The resulting linear time-invariant (LTI) model is valid only near the chosen operating point but enables all classical and modern control analysis tools. Professional flight control engineers design controllers at multiple operating points (called gain scheduling) to cover the full flight envelope.

The transfer function used in this project represents a typical aircraft at a specific cruise condition and is taken from the **University of Michigan** Control Tutorials for MATLAB

and Simulink (CTMS), one of the most widely referenced introductory aircraft control examples in undergraduate aerospace education.

4 Mathematical Model

4.1 Transfer Function

The linearized aircraft longitudinal pitch transfer function relating elevator deflection $\delta e(s)$ to pitch angle $\theta(s)$ is:

$$\frac{\theta(s)}{\delta_e(s)} = \frac{K(\tau s + 1)}{s^3 + a_2 s^2 + a_1 s + a_0}$$

With the following parameter values:

Table 1 : Aircraft Pitch Transfer Function Parameters

Parameter	Symbol	Value	Physical Meaning
Gain	K	-1.282	Control effectiveness (negative = nose down for positive δe)
Zero time constant	τ	-1.0	RHP zero location at $s =$ $+1/ \tau = +1$
Pitch damping coeff.	a_2	1.935	Aerodynamic pitch rate damping
Speed stability coeff.	a_1	0.987	Speed-pitch coupling derivative
Static stability coeff.	a_0	0.179	Static pitch restoring moment

Numerator Expansion:

$$K \times [\tau \ 1] = -1.282 \times [-1 \ 1] = [1.282 \ -1.282]$$

Numerical Transfer Function:

$$G(s) = \frac{1.282s - 1.282}{s^3 + 1.935s^2 + 0.987s + 0.179}$$

4.2 The Right-Half-Plane Zero

Setting the numerator to zero:

$$1.282s - 1.282 = 0 \rightarrow s = +1$$

This zero lies in the right-half plane (positive real part). This makes the system non-minimum phase. The physical consequence is that for a positive elevator command, the aircraft initially pitches downward before pitching upward — because the elevator deflection first creates a downward tail force, briefly rotating the nose down, before the moment builds and the nose pitches up. This initial wrong-direction response is called undershoot and is visible as a dip below zero at the start of every step response plot in this project.

The RHP zero imposes a hard upper bound on the closed-loop bandwidth and gain margin. No amount of PID tuning can eliminate the undershoot or significantly exceed this bandwidth limit.

5 MATLAB Open-Loop Analysis

5.1 Pole-Zero Analysis

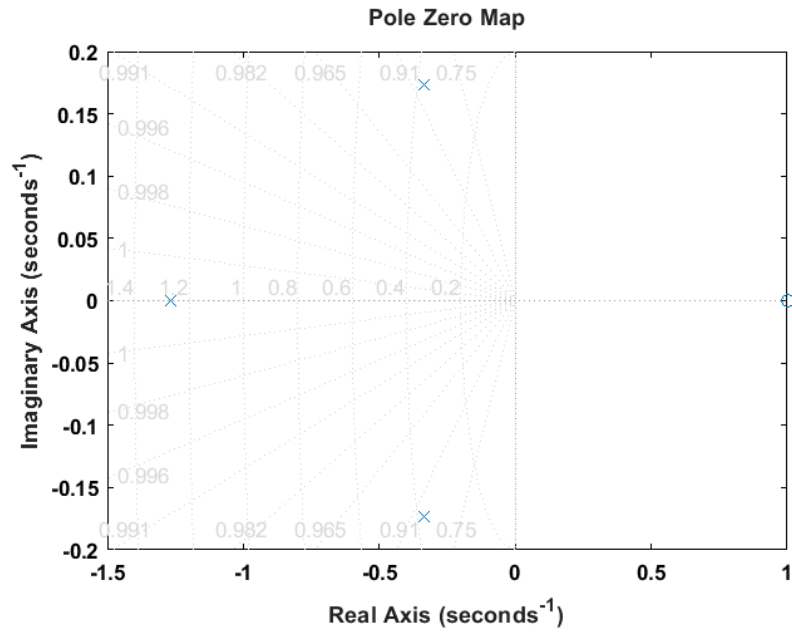


Figure 2 Pole-Zero Map — Aircraft Pitch Transfer Function

5.1.1 Poles:

Table 2 : Open-Loop Poles and Associated Flight Modes

Pole	Location	Type	Flight Mode
p_1	-1.2679	Real	Short Period Mode — fast, heavily damped
p_2	$-0.3336 + 0.1730j$	Complex	Phugoid Mode — slow, lightly damped
p_3	$-0.3336 - 0.1730j$	Complex	Phugoid Mode — conjugate pair

5.1.2 Zero:

$Z = +1$ (Right-Half Plane — Non-Minimum Phase)

5.1.3 Observation

All three poles are in the left-half plane, confirming open-loop stability. The Short Period mode at -1.2679 decays rapidly and governs the initial pitch response. The Phugoid mode complex poles at $-0.3336 \pm 0.1730j$ produce a slow, lightly damped oscillation where the aircraft periodically trades altitude for airspeed — a characteristic motion visible in all step response plots as a gentle oscillation before settling.

5.2 Open-Loop Step Response

5.2.1 Response

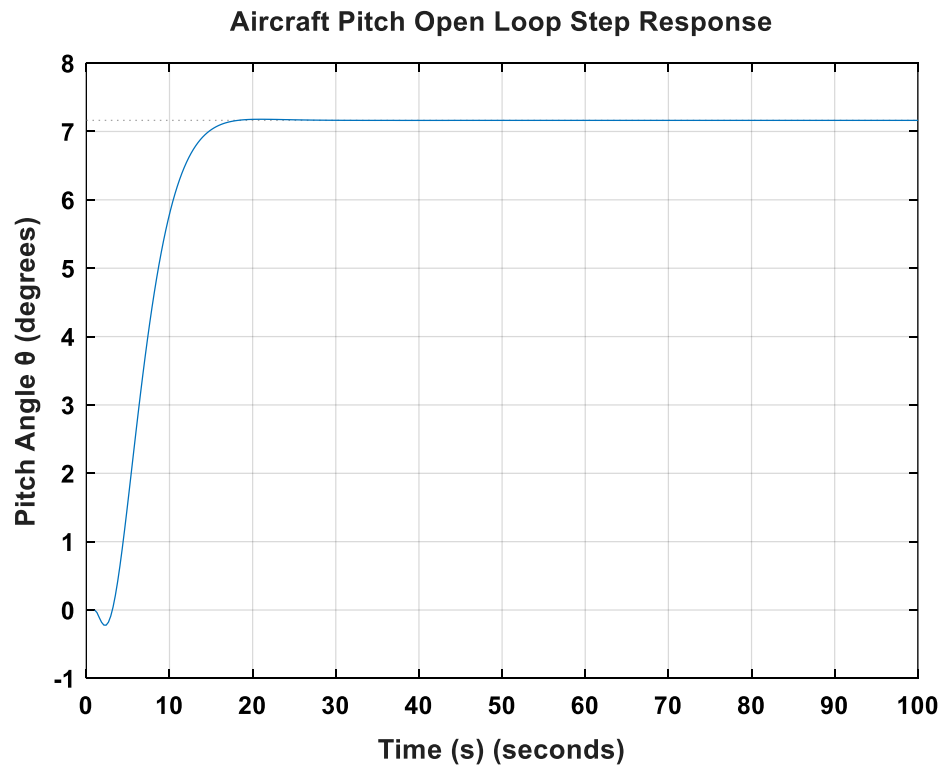


Figure 3 Aircraft Pitch Open-Loop Step Response — 1 Degree Elevator Deflection (MATLAB)

5.2.2 Performance Specification

Table 3: Open-Loop Step Response Performance Specifications

Parameter	Value
Rise Time	7.56 s
Settling Time	13.93 s
Overshoot	0.23%
Undershoot	3.16%
Steady-State Pitch Angle	-7.16 degrees (for +1° elevator)

5.2.3 Observation

For a positive elevator deflection of 1 degree, the aircraft pitches down to -7.16 degrees — confirming the negative gain $K = -1.282$. For the Simulink and closed-loop analysis, the sign convention was flipped ($K = +1.282$) so that a positive pitch command produces a positive pitch angle response, making the control loop physically intuitive. The undershoot of 3.16% is the RHP zero signature — the aircraft briefly pitches downward at $t \approx 0$ to 2s before the moment builds and drives the nose upward.

5.3 Sign Convention — Why K Was Flipped?

The original transfer function has $K = -1.282$, meaning positive elevator input produces negative pitch. For closed-loop design, this creates unintuitive sign conventions in the feedback loop. By using $K = +1.282$ in the Simulink implementation, a positive reference pitch command (e.g., +5 degrees) produces a positive pitch response. This is equivalent to what was done in MATLAB with $\text{step}(-G)$ and is standard practice in flight control system design — the plant sign convention is absorbed into the implementation, so the controller always operates on intuitive positive error signals.

6 . Parameter Sensitivity Study

The three denominator coefficients a_0 , a_1 , and a_2 were each varied independently while the remaining parameters were held at nominal values. For each variation, the poles and performance specifications were recorded and physically interpreted.

6.1 Effect of Static Stability (a_0)

The static stability coefficient a_0 represents the **restoring pitch moment** — **how strongly the aircraft tends to return to its original pitch when disturbed**. A larger a_0 means a more statically stable aircraft.

6.1.1 Response

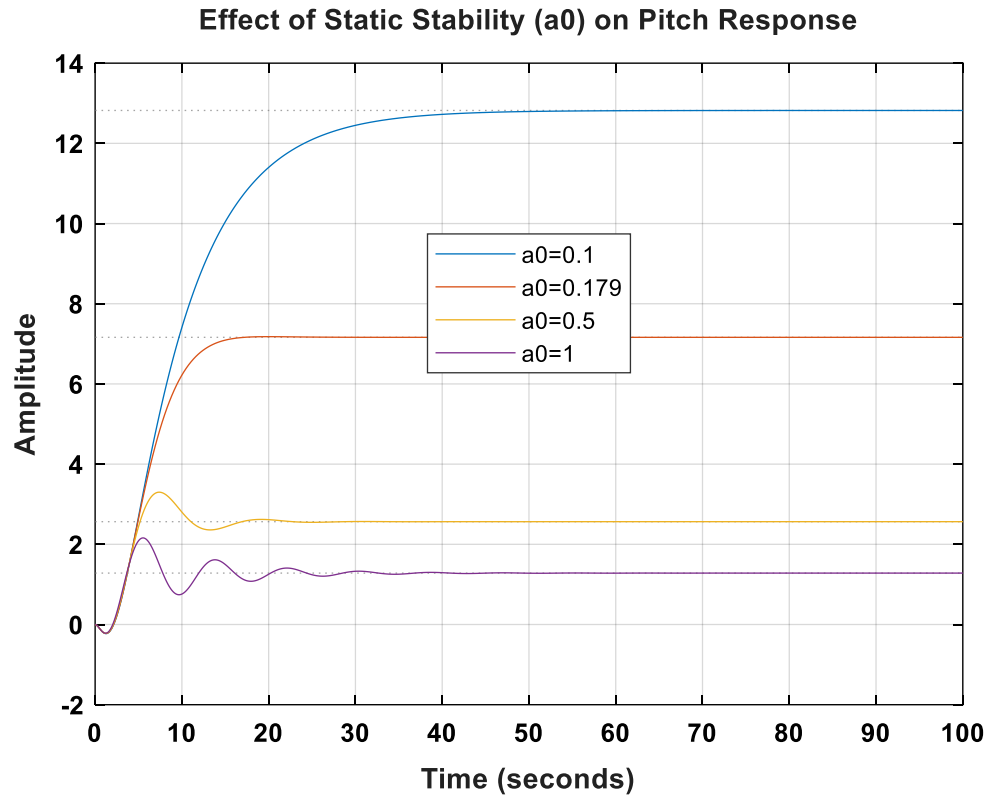


Figure 4 Effect of Static Stability Coefficient (a_0) on Pitch Step Response

6.1.2 Poles and Performance Specifications

Table 4: Effect of a_0 on Pole Locations and Performance

a_0	Poles	Nature	Overshoot	Settling Time	SS Pitch
0.1	-1.156, -0.645, -0.134	Real — overdamped	0%	32.76 s	-12.82°
0.179 (nominal)	-1.268, -0.334±0.173j	One real + complex pair	0.23%	13.93 s	-7.18°
0.5	-1.499, -0.218±0.535j	One real + complex pair	28.71%	19.96 s	-3.30°
1.0	-1.700, -0.117±0.758j	One real + complex pair	68.44%	35.20 s	-2.16°

6.1.3 Observations:

- At small a_0 the three poles are all real and negative — the system is overdamped with no oscillation.
- As a_0 increases, two poles become complex conjugates. The imaginary part of these complex poles grows as a_0 increases, causing more oscillation.
- The steady-state pitch angle decreases as a_0 increases because greater static stability means the aircraft resists elevator-induced pitch changes more strongly.
- The non-monotonic settling time behavior is explained by the transition between overdamped and underdamped regimes — once complex poles appear, settling time is governed by their real part, not by the overdamped decay rate.

6.2 Effect of Pitch Damping (a_2)

The pitch damping coefficient a_2 represents aerodynamic resistance to pitch rate — how strongly the tail damps the rotational motion. Higher a_2 is analogous to higher viscous friction in the DC motor.

6.2.1 Response

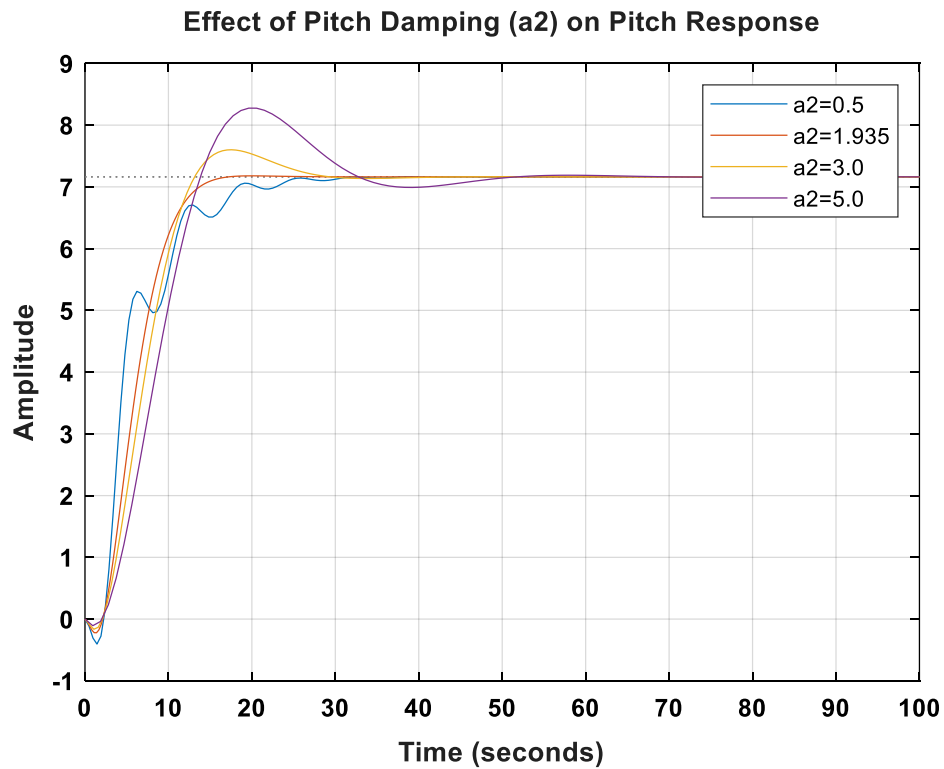


Figure 5 Effect of Pitch Damping Coefficient (a_2) on Pitch Step Response

6.2.2 Poles and Performance Specifications

Table 5: Effect of a_2 on Pole Locations and Performance

a_2	Poles	Overshoot	Settling Time	SS Pitch
0.5	$-0.154 \pm 0.951j$, -0.193	0.01%	23.17 s	Independent
1.935 (nominal)	-1.268 , $-0.334 \pm 0.173j$	0.23%	13.93 s	Independent
3.0	-2.654 , $-0.173 \pm 0.194j$	6.13%	24.46 s	Independent
5.0	-4.802 , $-0.099 \pm 0.166j$	15.60%	42.37 s	Independent

6.2.3 Observations

- Increasing pitch damping actually increased overshoot and settling time at high values. This is counterintuitive but physically explained by the dominant pole effect.
- As a_2 increases, one pole moves far to the left (decays very fast) while the complex pair moves closer to the imaginary axis (decays very slowly). The slow complex poles become dominant — governing the response — and because their real part is very small at high a_2 , the system oscillates more persistently.
- The **a_2 paradox** is a real aircraft design trade-off: more aerodynamic pitch damping from a larger tail does not always produce a better-behaved response.

6.3 Effect of Dynamic Stability (a_1)

The coefficient a_1 appears in the s^1 term — analogous to the damping term b in the DC motor denominator. It couples the speed and pitch dynamics and represents the effect of angle-of-attack changes on pitch rate.

6.3.1 Response

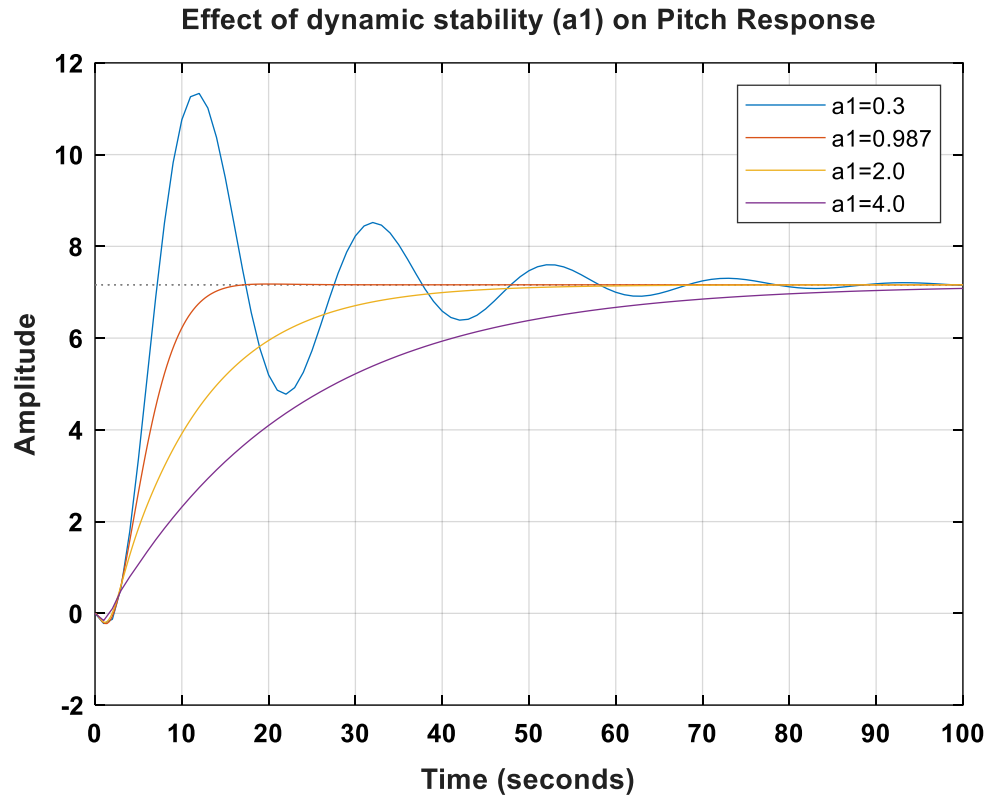


Figure 6 Effect of Dynamic Stability Coefficient (a_1) on Pitch Step Response

6.3.2 Poles and Performance Specifications

Table 6: Effect of a_1 on Pole Locations and Performance

a_1	Poles	Overshoot	Settling Time	SS Pitch
0.3	$-1.824,$ $-0.055 \pm 0.308j$	58.56%	65.88 s	Independent
0.987 (nominal)	$-1.268,$ $-0.334 \pm 0.173j$	0.23%	13.93 s	Independent
2.0	$-0.918 \pm 0.988j,$ -0.098	0%	41.72 s	Independent
4.0	$-0.945 \pm 1.738j,$ -0.046	0%	86.99 s	Independent

6.3.3 Observations:

- Increasing a_1 moves the dominant pole closer to the imaginary axis, making the system slower even though it appears more damped.
- Steady-state pitch angle is independent of a_1 , confirming it only affects transient dynamics.
- At low a_1 , the complex poles have large imaginary parts relative to their real parts, producing significant oscillation.

6.4 Parameter Summary

Table 7: Summary of Parameter Sensitivity Effects

Parameter	Physical Meaning	Effect on SS Pitch	Effect on Oscillation	Effect on Settling
a0 increases	More statically stable	Decreases	Increases	Non-monotonic
a1 increases	More speed-pitch coupled	Independent	Decreases initially	Increases
a2 increases	More pitch rate damped	Independent	Paradox — increases at high a2	Increases at high a2

7 Simulink Open-Loop Verification

7.1 Block Diagram

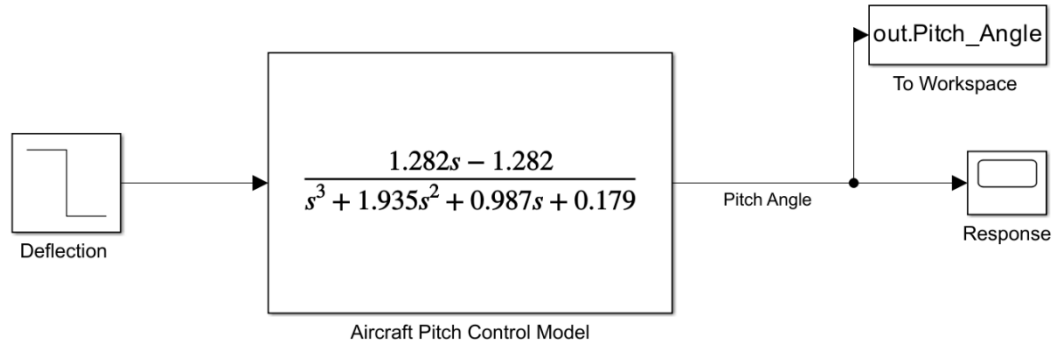


Figure 7 Simulink Block Diagram — Open-Loop Aircraft Pitch Model

7.1.1 Response

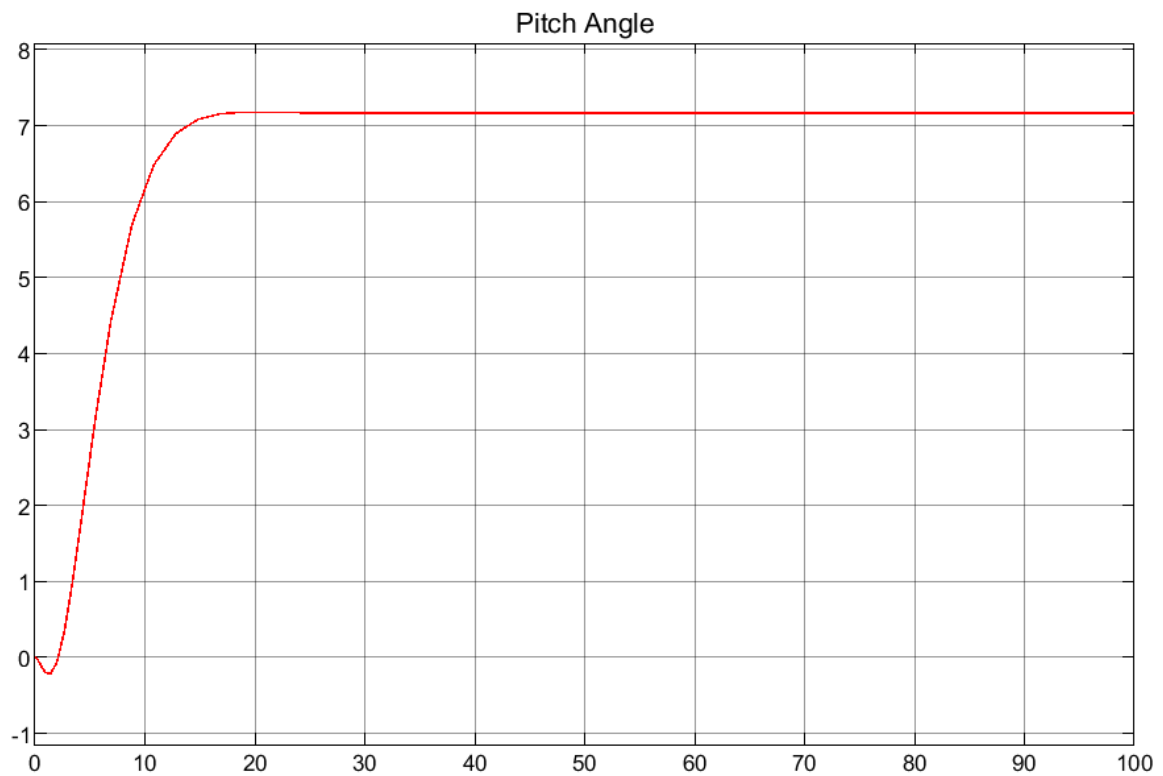


Figure 8 Open-Loop Step Response — Simulink Scope (Matches MATLAB)

7.1.2 Observations

- The Simulink open-loop model was built using the transfer function with numerator $[1.282 \ -1.282]$ and denominator $[1 \ 1.935 \ 0.987 \ 0.179]$, with a step input of -1 (to produce positive pitch with the original negative-gain plant).
- The scope output matched the MATLAB $\text{step}(-G)$ result exactly: undershoot visible at $t = 0$ to $2s$, settling to $+7.16$ degrees, same oscillation shape.
- This confirms the Simulink model is correctly implemented and ready for closed-loop controller addition.

8 . Closed-Loop PID Control

For all closed-loop simulations, the numerator sign was flipped to $K = +1.282$, making the plant positive-gain so that a +5 degree pitch command produces a +5 degree response in a physically intuitive feedback loop. The reference command was set to 5 degrees — a realistic autopilot pitch-up command.

8.1 P Controller Analysis

8.1.1 Simulink Block Diagram

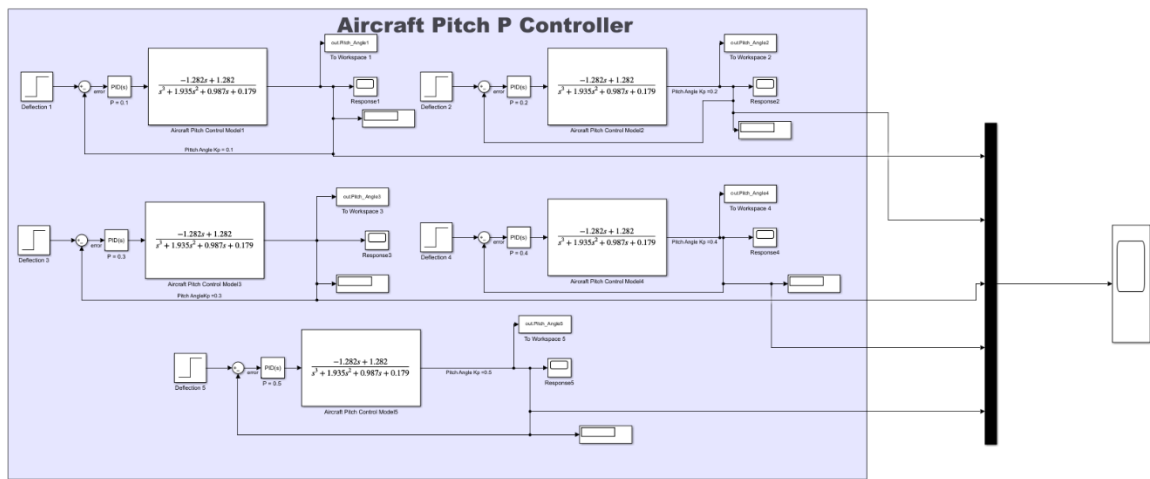


Figure 9 Simulink Block Diagram — P Controller (Five Kp Values)

8.1.2 Responses — 30s and 100s Simulation

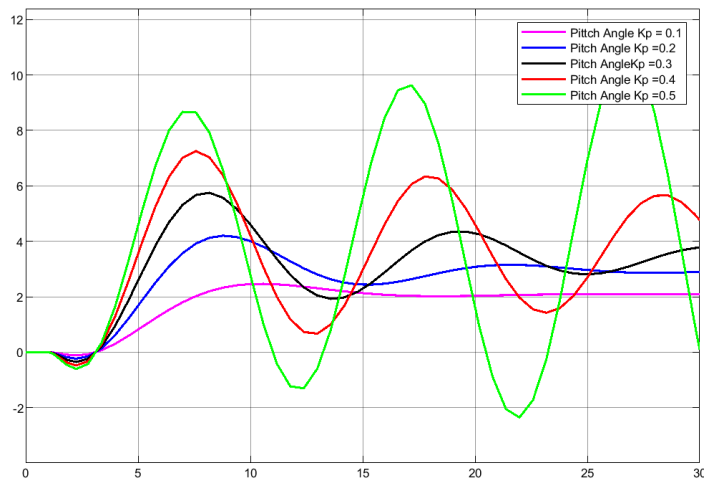


Figure 10 P Controller Step Responses for $K_p = 0.1, 0.2, 0.3, 0.4, 0.5$ (30s simulation)

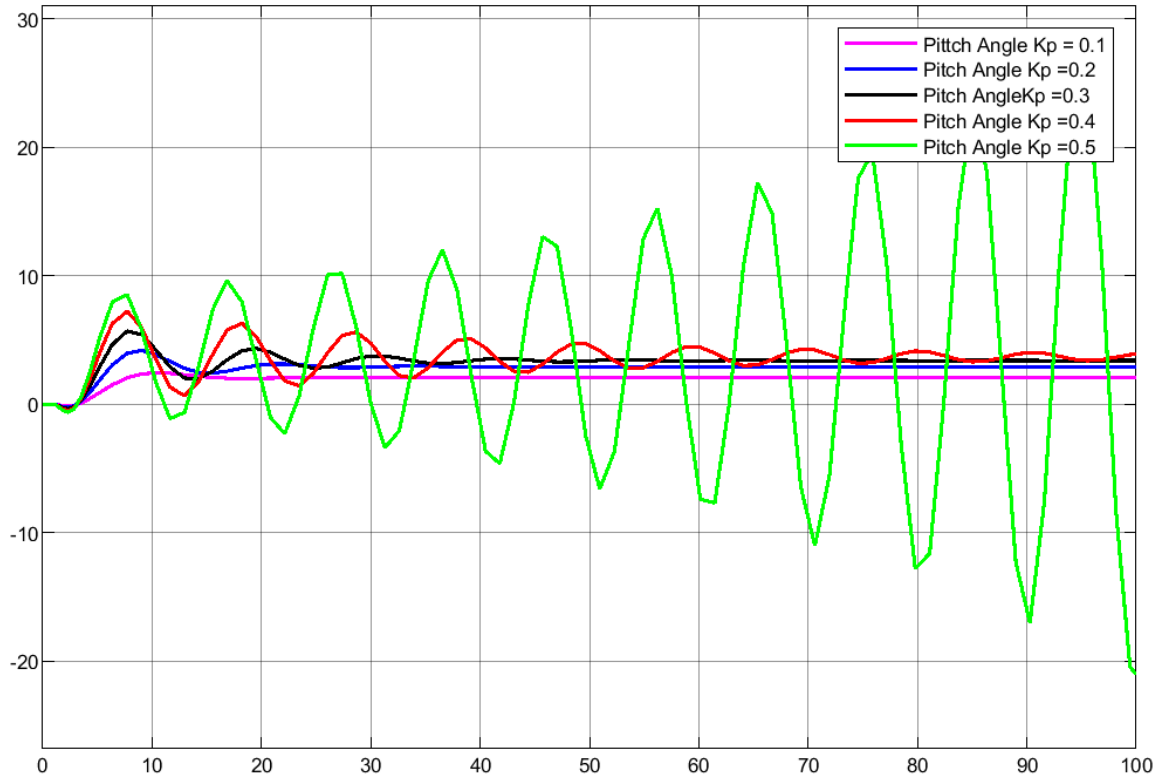


Figure 11 P Controller Step Responses for $K_p = 0.1, 0.2, 0.3, 0.4, 0.5$ (100s simulation)

8.1.3 Performance Specifications

Table 8: P Controller Performance — K_p Variation (100s simulation)

K_p	Rise Time	Settling Time	Overshoot	Steady-State (deg)	Stable?
0.1	3.51 s	20.73 s	18.12%	~2.09	Yes
0.2	2.36 s	29.54 s	42.52%	~2.90	Yes
0.3	1.80 s	48.90 s	67.33%	Marginal	Marginal
0.4	1.55 s	99.30 s	84.37%	Barely stable	Edge
0.5	N/A	N/A	N/A	Diverging	UNSTABLE

8.1.4 Gain Margin Discovery:

The system becomes unstable between $K_p = 0.4$ and $K_p = 0.5$. This is the gain margin of the system. At $K_p = 0.1$, the pitch settles at only 2.09 degrees instead of the commanded 5 degrees — a steady-state error of 58%. **The problem is that this error cannot be corrected by increasing K_p because instability appears before K_p is large enough to reduce SSE significantly.**

8.1.5 Observations

- This behavior is fundamentally different from the DC motor in Project 03, where K_p could be pushed to 50 without instability. The RHP zero in the aircraft plant sharply limits the allowable loop gain. This is exactly why aircraft autopilots require integral action — not just to eliminate SSE, but because P control hits the stability boundary before achieving acceptable performance.
- An important lesson about **simulation time**: results at 30 seconds showed $K_p = 0.4$ appearing stable, while at 100 seconds the same simulation revealed growing oscillations. ***Simulation time must be at least $5\times$ the expected settling time to avoid false stability conclusions near the gain margin.***

8.2 PI Controller Analysis

8.2.1 Simulink Block Diagram

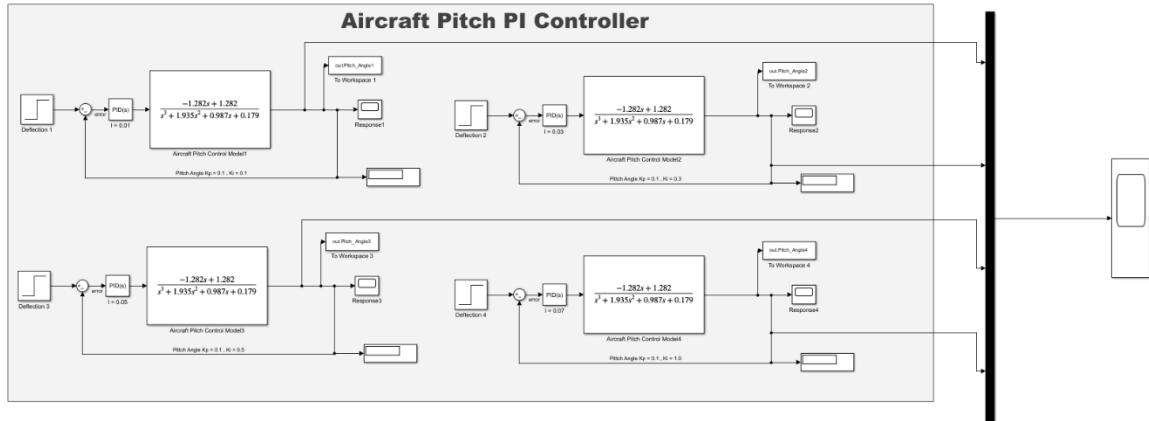


Figure 12 Simulink Block Diagram — PI Controller

8.2.2 Step Responses

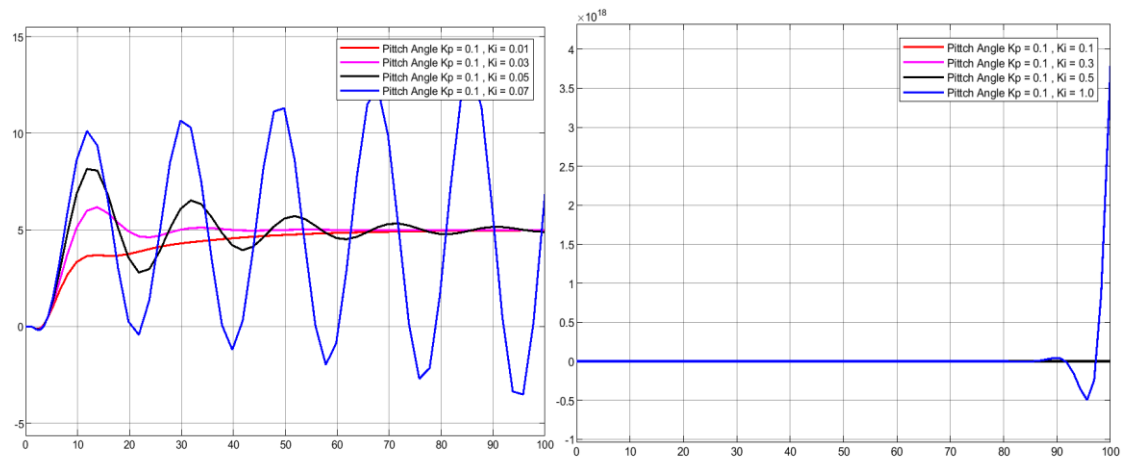


Figure 13 PI Controller Step Responses — $K_i = 0.01, 0.03, 0.05, 0.07$ ($K_p=0.1, 100s$)

8.2.3 Performance Specifications

Table 9: PI Controller Performance — Ki Variation ($K_p=0.1$, 100s)

Ki ($K_p=0.1$)	Settling Time	SSE	Peak	Stable?
0.01	65.31 s	0% — Peak = 4.98 $\approx 5^\circ$	4.98°	Yes
0.03	35.04 s	0% — overshoots first	6.17°	Yes, but overshoots
0.05	96.14 s	Marginally unstable	8.15°	Marginal
0.07	Unstable	—	13.42°	UNSTABLE

Observations:

- Integral action successfully eliminates steady-state error. At $K_i=0.01$, the aircraft reaches exactly 5 degrees — zero SSE — which P control alone could never achieve.
- However, the settling time of 65 seconds is unacceptably slow for a real aircraft autopilot (which must respond in 2-5 seconds).
- The stability boundary for K_i is extremely tight — the system becomes unstable at $K_i=0.05$ to 0.07 with $K_p=0.1$. This confirms the fundamental constraint:
“the RHP zero limits both proportional and integral gains, making it impossible to simultaneously achieve zero SSE and fast response with a simple PI controller.”
- The slow response is not a **tuning failure** — it is a consequence of the plant physics. Real aircraft autopilots address this by using **cascade control architectures**: an outer loop commanding pitch rate rather than pitch angle, with an inner loop controlling elevator deflection.
- This separates the non-minimum phase dynamics from the integrator and allows much more aggressive tuning.

8.3 PID Automatic Tuning — pidtune()

MATLAB's `pidtune()` function was applied to the positive-gain aircraft transfer function to find the best achievable PID gains:

8.3.1 Response

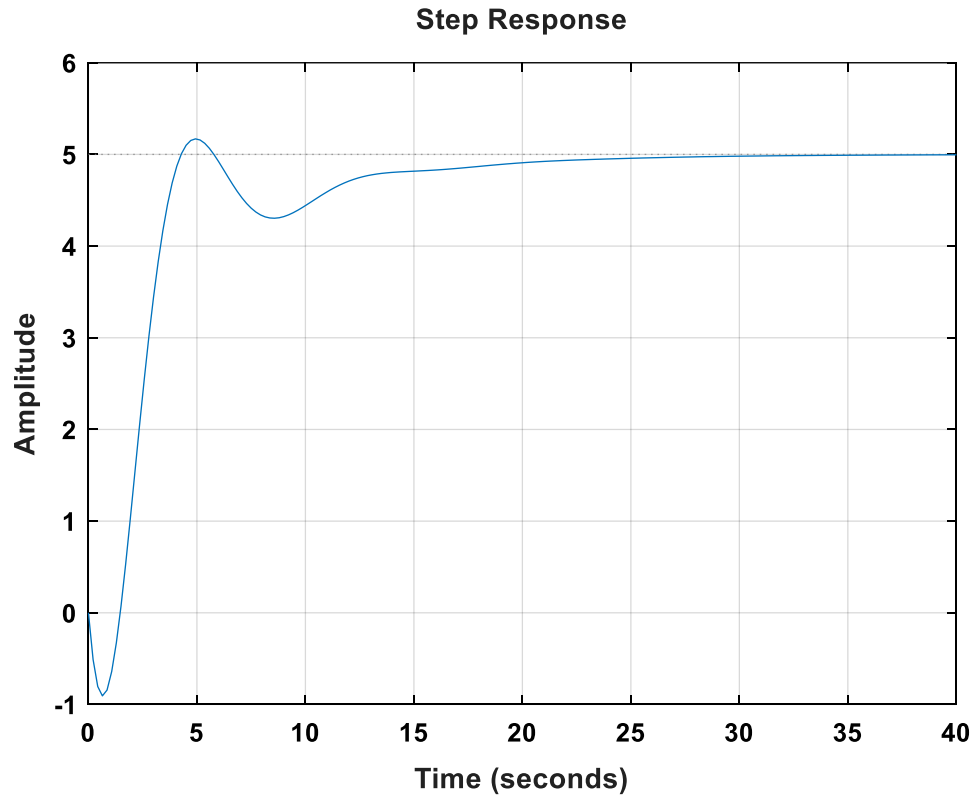


Figure 14 Automatic PID Tune Step Response

8.3.2 Performance Specifications

Table 10: `pidtune()` Automatic PID Gains and Performance

	Kp	Ki	Kd	Rise Time	Settling Time	Overshoot	Undershoot
pidtune() Result	0.264	0.0372	0.455	1.99 s	19.49 s	3.39%	18.12%

8.3.3 Analysis of pidtune() Gains:

- Notice that all three gains are small — pidtune() could not use aggressive gains without instability. $K_p=0.264$ is only slightly higher than the manually tested $K_p=0.1$, and $K_i=0.0372$ is in the same range as the stable $K_i=0.01$ found manually. The $K_d=0.455$ provides derivative action to partially compensate for the RHP zero's destabilizing effect on the phase margin.
- The 18.12% undershoot is the RHP zero signature — it cannot be reduced below a minimum value regardless of tuning.
- The settling time of 19.5 seconds, while much faster than the manual PI result, is still far slower than a real aircraft autopilot requires.
- **This confirms that PID is not the appropriate controller architecture for non-minimum phase aerospace plants.**

9 Controller Comparison

Table 11: Complete Controller Comparison — Aircraft Pitch System

Controller	Rise Time	Settling Time	Overshoot	Undershoot	SS Error	Stable?
Open Loop (1° input)	7.56 s	13.93 s	0.23%	3.16%	Large (gain dependent)	Yes
P ($K_p=0.1$)	3.51 s	20.73 s	18.12%	5.46%	58%	Yes
P ($K_p=0.5$)	N/A	N/A	N/A	N/A	N/A	UNSTABLE
PI ($K_p=0.1$, $K_i=0.01$)	31.90 s	65.31 s	0%	2.47%	0%	Yes
PID — pidtune()	1.99 s	19.49 s	3.39%	18.12%	0%	Yes

9.1.1 Observations

- The comparison reveals the fundamental challenge of non-minimum phase control: achieving zero SSE requires integral action, but integral action combined with even modest proportional gain pushes the system toward instability.
- The best achievable PID performance (pidtune result) gives 19.5 seconds settling time — far slower than the 2-5 seconds required for a real aircraft autopilot.
- This performance gap motivates the state-space methods in Projects 07-09, where LQR on the same plant will achieve settling times under 5 seconds.

10 Disturbance Rejection — Wind Gust Test

A step disturbance of +2 degrees was injected at the plant input at $t=50s$, simulating a sudden wind gust that perturbs the aircraft pitch after the autopilot has already settled at the commanded 5 degrees. The `pdtune()` gains ($K_p=0.264$, $K_i=0.0372$, $K_d=0.455$) were used.

10.1 Simulink Block Diagram

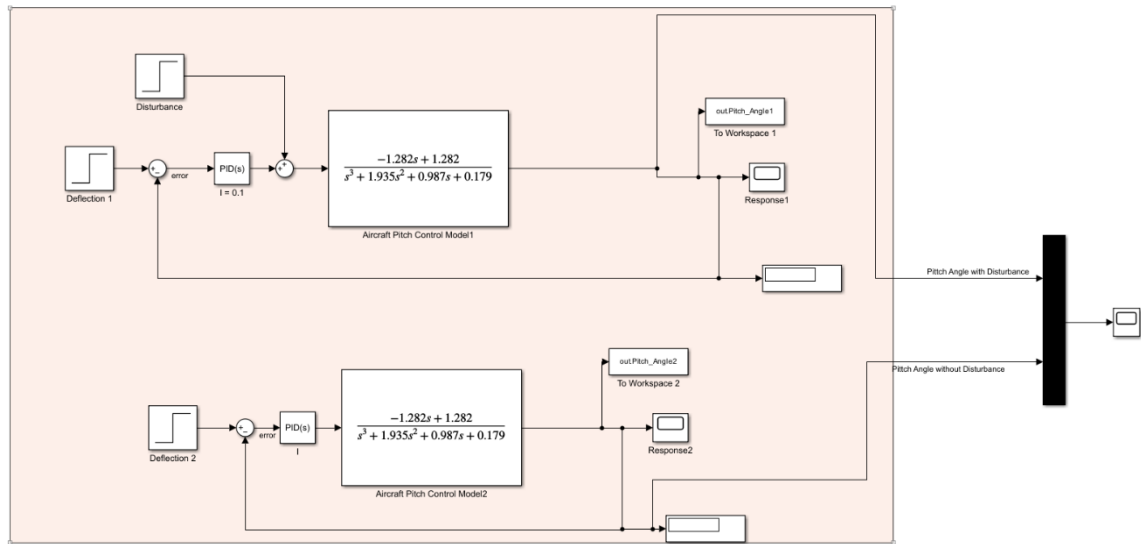


Figure 15 Simulink Block Diagram — Wind Gust Disturbance Rejection Test

10.2 Disturbance Rejection Response

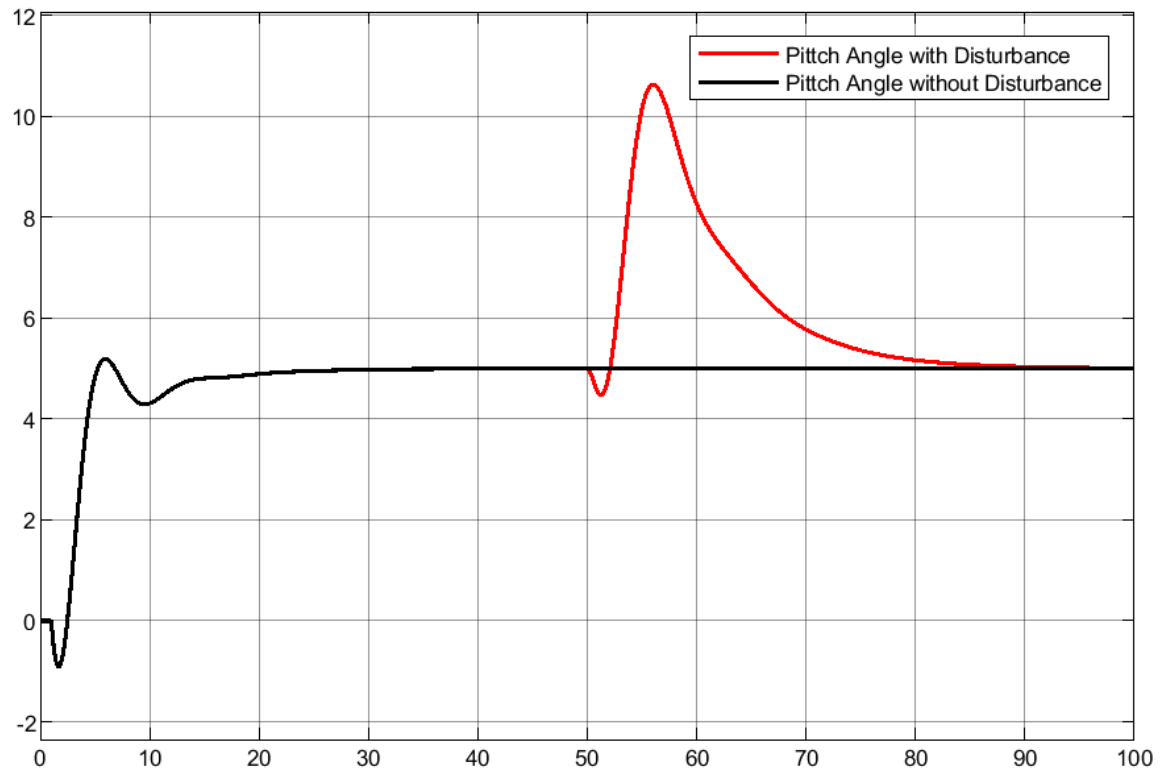


Figure 16 Aircraft Pitch Response With and Without 2-Degree Wind Gust at $t=50s$

10.2.1 Rejection Results

Table 12: Wind Gust Disturbance Rejection Results

Condition	Pitch at $t=50s$	Peak After Disturbance	Final Pitch	Recovery
Without disturbance	5.00°	—	5.00°	N/A
With $+2^\circ$ gust at $t=50s$	5.00°	$\sim 10.6^\circ$	5.00°	Complete

10.3 Observations:

- The PID autopilot successfully rejected the 2-degree wind gust disturbance and returned the aircraft to exactly 5 degrees commanded pitch by approximately $t=83s$.
- The pitch spiked to 10.6 degrees immediately after the disturbance — the controller detected this as an error (reference= 5° , output= 10.6° , error= -5.6°), reduced elevator input, and gradually brought the aircraft back to the commanded pitch.
- The slow recovery time of 33 seconds after the disturbance is again a consequence of the tight gain margins imposed by the RHP zero. A real autopilot would use a more sophisticated control structure to recover faster.
- The disturbance rejection result is not a failure. The integral term ensures that the aircraft returns to exactly the commanded pitch — zero steady-state error is maintained even after the disturbance. An open-loop system or P-only controller would settle at a permanently different pitch angle after the gust.

11 Aerospace Relation

The aircraft pitch control problem studied in this project is not an academic exercise — it represents the exact class of control challenge faced by flight control engineers at Boeing, Airbus, Lockheed Martin, and every modern aircraft manufacturer.

Commercial Aircraft Autopilots: Modern fly-by-wire aircraft such as the Airbus A320 and Boeing 787 use digital flight control computers that continuously compute and apply control surface commands. The pitch autopilot must maintain commanded pitch angles during cruise, respond to turbulence, and execute precise pitch maneuvers during approach and landing — all while respecting structural load limits. **The gain margin limitation discovered in this project directly explains why real autopilots use complex multi-loop architectures rather than simple PID.**

Unmanned Aerial Vehicles (UAVs): Fixed-wing UAVs such as the MQ-9 Reaper or Boeing ScanEagle use pitch autopilots to maintain flight paths during reconnaissance missions. The non-minimum phase behavior of the elevator means the autopilot must be carefully tuned to avoid exciting the phugoid mode during altitude changes — exactly the slow oscillation observed in the Phugoid mode poles of this project.

Teknofest VLR Rocket: The vertical landing rocket uses pitch and roll control for attitude stabilization during descent. While the rocket dynamics differ from aircraft (no phugoid mode, changing mass due to fuel consumption), the same fundamental challenge applies: the control bandwidth is limited by stability margins, and the controller must reject atmospheric disturbances while maintaining commanded attitude. The gain margin concepts and disturbance rejection methodology in this project apply directly.

Why PID is Not Enough for Real Aircraft: Real flight control systems use cascaded architectures with inner pitch rate loops and outer pitch angle loops, gain scheduling across the flight envelope, notch filters to avoid exciting structural modes, and model-following control laws. These sophisticated architectures all build on the same fundamental understanding developed in this project — and all of them were motivated by exactly the limitations of simple PID that this project demonstrates.

12 Conclusions

This project successfully completed the mathematical modeling, open-loop analysis, parameter sensitivity study, and closed-loop PID control of an aircraft longitudinal pitch dynamics system. The following engineering conclusions were established:

1. The aircraft pitch system has two fundamental longitudinal modes. The Short Period mode (pole at -1.2679) provides fast, heavily damped pitch response. The Phugoid mode (complex poles at $-0.3336 \pm 0.1730j$) produces slow, lightly damped oscillation where the aircraft trades altitude for airspeed.
2. The RHP zero at $s = +1$ makes this a non-minimum phase system. The physical cause is the elevator's initial downward tail force, which briefly pitches the aircraft in the wrong direction before the moment builds to pitch it correctly. This undershoot (3.16% in open loop) cannot be eliminated by any PID controller.
3. Static stability (a_0) affects steady-state pitch angle. Increasing a_0 makes the aircraft more stable but reduces the steady-state pitch achieved for a given elevator input. Pitch damping (a_2) and dynamic stability (a_1) do not affect steady-state pitch but strongly influence transient behavior.
4. P control alone is fundamentally inadequate for this plant. The system becomes unstable between $K_p = 0.4$ and $K_p = 0.5$. At the maximum stable $K_p = 0.1$, the steady-state error is 58%. P control cannot simultaneously satisfy stability and acceptable SSE requirements for a non-minimum phase plant.
5. Integral action eliminates steady-state error but with a very tight stability margin. PI control with $K_i = 0.01$ achieved exactly 5 degrees pitch with zero SSE, but required 65 seconds to settle — unacceptable for real autopilot applications. The stability boundary for K_i was between 0.03 and 0.05.
6. The best achievable PID performance (pidtune: $K_p=0.264$, $K_i=0.0372$, $K_d=0.455$) achieved 19.5 seconds settling time with 3.39% overshoot and 18.12% undershoot. This confirms that the RHP zero imposes a fundamental performance limitation that cannot be overcome by PID gain selection alone.
7. The PID autopilot successfully rejected a 2-degree wind gust disturbance, returning the aircraft to exactly 5 degrees commanded pitch. Disturbance rejection is the most

practically important property of the closed-loop system, and the integral term ensures zero steady-state error is maintained despite external perturbations.

8. PID is not the right controller for non-minimum phase aerospace plants. The gain margin is too tight for aggressive tuning, and settling times are inherently slow. State-space methods (Projects 07-09) will demonstrate how LQR can achieve 5-second settling times on the same plant — a $4\times$ improvement over the best PID result.

13 Future Work

Project 04 revealed the fundamental limitations of PID control on non-minimum phase aerospace plants. The following projects address these limitations using progressively more powerful control design methods:

Project 05 — Root Locus Design

Formal root locus analysis will replace the trial-and-error gain tuning used in this project. By plotting how closed-loop poles move as a function of controller gain, it becomes possible to select gains analytically — predicting exactly what rise time and damping ratio will result from a given K_p before running a single simulation.

Project 06 — Bode and Nyquist Analysis

Gain margin and phase margin will be computed to formally characterize how much parameter variation or time delay the closed-loop system can tolerate before becoming unstable. These stability margins are a mandatory design requirement in aerospace control systems and cannot be assessed from step response alone.

Project 07 — State Space Modeling

The DC motor and aircraft plants will be re-modeled in state space form using the A, B, C, D matrix representation. State space is the language of modern control theory and is required for all subsequent advanced controller designs. It also enables simulation of multi-input multi-output systems, which are essential for full aircraft and rocket control.

Project 08 — Pole Placement

Full state feedback will be used to place closed-loop poles at precisely specified locations in the complex plane, achieving exact desired transient performance. Unlike PID tuning, pole placement provides a direct mathematical relationship between desired performance specifications and required controller gains.

Project 09 — LQR Optimal Control

The Linear Quadratic Regulator will be designed by minimizing a cost function that balances control effort against state error. LQR produces optimal state feedback gains automatically and is one of the most widely used controllers in aerospace applications including aircraft autopilots, satellite attitude control, and rocket landing systems.

Project 10 — Kalman Filter

A Kalman filter will be designed to estimate system states from noisy sensor measurements. In real hardware, IMU sensors on rockets and UAVs produce noisy angular rate and acceleration data. The Kalman filter fuses this noisy data with a system model to produce clean state estimates — it is the essential bridge between simulation and physical hardware implementation, and is directly required for the Teknofest VLR rocket.

Project 11 — UAV Attitude Control

A full quadrotor attitude control system will be built in Simulink, including separate PID controllers for roll, pitch, and yaw axes, motor mixing logic, and nonlinear rigid body dynamics. This project introduces multi-axis control and the coupling between axes that does not exist in single-input single-output systems.

Project 12 — Rocket Attitude Control

A vertical landing rocket attitude control system — directly modeled on the Teknofest VLR architecture — will be simulated in Simulink. This includes cold gas RCS thruster modeling with PWPF modulation, TVC gimbal dynamics, cascaded PID loops for attitude and position, and 6DOF rigid body dynamics. This project is the direct simulation validation tool for the actual competition hardware.

Project 13 — Satellite Attitude Control

Reaction wheel based attitude control using quaternion kinematics will be implemented. Satellite control introduces zero-gravity dynamics, conservation of angular momentum constraints, and the requirement for propellant-free maneuvering — all of which require modern control techniques beyond classical PID.

Project 14 — Missile Guidance

Proportional navigation guidance law will be implemented and analyzed. Missile guidance connects control theory to trajectory optimization and introduces the concept of commanded acceleration as a control input, which differs fundamentally from the position and speed control studied in earlier projects.

Project 15 — Complete Flight Control System

A complete integrated flight control system will be assembled combining autopilot, attitude control, guidance, and navigation into a single Simulink environment. This capstone project represents the synthesis of all fifteen projects and produces a portfolio piece demonstrating end-to-end aerospace control system design capability.

14 Appendix

14.1 MATLAB Code

14.1.1 Open Loop Step Response

```
%% ----- Open Loop Step Response of Aircraft Pitch Control-----
clc ; clear all ; close all ;

%% System Parameters
K = -1.282;
tau = -1.0;
a2 = 1.935;
a1 = 0.987;
a0 = 0.179;
%% Transfer Function
num = [K*tau K];
den = [1 a2 a1 a0];
G = tf(num,den);
%% Analysis

disp("----Poles----")
P = pole(G)

disp("----Zeros----")
Z = zero(G)

disp("----System Performance----")
S = stepinfo(G)

%% Plots

figure(1)
step(-G,[1 100])
title("Aircraft Pitch Open Loop Step Response")
xlabel("Time (s)");
ylabel("Pitch Angle  $\theta$  (degrees)")
grid on

figure(2)
pzmap(G)
title("Pole Zero Map")
grid on

%% ----- Parameter Study -----
```

```

K = -1.282;
tau = -1.0;
a2 = 1.935;
a1 = 0.987;
a0 = 0.179;

%% Effect of a0 (static stability)

figure(3)
a0_values = [0.1, 0.179, 0.5, 1];
for i = 1 : length (a0_values)
    a0 = a0_values(i);
    G_a0 = tf([K*tau K] , [1 a2 a1 a0]);
    step(-G_a0,100)
    hold on
    p = pole(G_a0);
    fprintf("Poles of system when a2 , a1 , K , tau are constant
and a0 = %.3f :\n",a0)
    disp(p)
    info_a0 = stepinfo(G_a0);
    fprintf("System Performance when a2 , a1 , K , tau are
constant and a0 = %.3f\n",a0)
    disp(info_a0)
end
legend("a0=0.1","a0=0.179","a0=0.5","a0=1",'Location','best')
title("Effect of Static Stability (a0) on Pitch Response")
grid on

```

```

%% Effect of a2 (pitch damping)

figure(4)
a2_values = [0.5, 1.935, 3.0, 5];
for i = 1 : length (a2_values)
    a2 = a2_values(i);
    G_a2 = tf([K*tau K] , [1 a2 a1 a0]);
    step(-G_a2,100)
    hold on
    p = pole(G_a2);
    fprintf("Poles of system when a0 , a1 , K , tau are constant
and a2 = %.3f :\n",a2)
    disp(p)
    info_a2 = stepinfo(G_a2);
    fprintf("System Performance when a0 , a1 , K , tau are
constant and a2 = %.3f\n",a2)

```

```

        disp(info_a2)
    end
    legend("a2=0.5", "a2=1.935", "a2=3.0", "a2=5.0")
    title("Effect of Pitch Damping (a2) on Pitch Response")
    grid on

%% Effect of a1 (dynamic stability)

figure(5)
a1_values = [0.3, 0.987, 2.0, 4.0];
for i = 1 : length (a1_values)
    a1 = a1_values(i);
    G_a1 = tf([K*tau K] , [1 a2 a1 a0]);
    step(-G_a1,100)
    hold on
    p = pole(G_a1);
    fprintf("Poles of system when a0 , a2 , K , tau are constant  
and a1 = %.3f :\n",a1)
    disp(p)
    info_a1 = stepinfo(G_a1);
    fprintf("System Performance when a0 , a2 , K , tau are  
constant and a1 = %.3f\n",a1)
    disp(info_a1)
end
legend('a1=0.3', 'a1=0.987', 'a1=2.0', 'a1=4.0')
title("Effect of dynamic stability (a1) on Pitch Response")
grid on

```

14.1.2 Closed Loop Step Response

```
%% ----- Closed Loop System Performance Anlysis of Aircraft
Pitch Control System-----
clc

%% ----- Closed Loop Aircraft Pitch Control System with only P
controller-----

kp_value = [0.1,0.2,0.3,0.4,0.5];
for i = 1:length(kp_value)
    signal = out.(sprintf('Pitch_Angle%d',i));
    info = stepinfo(signal.Data, signal.Time);
    fprintf("System Performance when Kp = %.1f is
:\n",kp_value(i))
    disp(info)
end

%% ----- Closed Loop Aircraft Pitch Control System with only
PI controller-----

ki_value = [0.1,0.3,0.5,1.0];
for i = 1:length(ki_value)
    signal = out.(sprintf('Pitch_Angle%d',i));
    info = stepinfo(signal.Data, signal.Time);
    fprintf("System Performance when Kp = 0.1 and Ki = %.1f is
:\n",ki_value(i))
    disp(info)
end
%% Auto Tune
K=1.282; tau=-1.0;
num = K*[tau 1];
den = [1 1.935 0.987 0.179];
G = tf(num,den);

C = pidtune(G,'PID')
T = feedback(C*G,1);
stepinfo(T)
step(5*T)

%% ----- Disturbance Rejection Case ---

kp = 0.264 ; ki = 0.0372 ; kd = 0.455 ;
```

```

% Step Disturbance of value 2 degrees at t = 50s
info1 = stepinfo(out.Pitch_Angle1.Data, out.Pitch_Angle1.Time);
fprintf("System Performance when Disturbance of 2 degrees is
involved:\n")
disp(info1)

info2 = stepinfo(out.Pitch_Angle2.Data, out.Pitch_Angle2.Time);
fprintf("System Performance without the Disturbance is :\n")
disp(info2)

```

14.1.3 Animation with Disturbance

```

%
=====
% Production-Ready Aircraft Pitch Animation & Video Recorder
% Features: Window-close protection, automatic export to MP4
video file
%
=====
clear; clc; close all;

%% 1. Load and Extract Real Simulink Data Robustly
if exist('out','var')
    t_sim = out.tout;
    pitch_raw = out.pitch_angle;

    if isa(pitch_raw, 'timeseries')
        theta_deg = pitch_raw.Data;
    elseif isa(pitch_raw, 'struct') && isfield(pitch_raw,
'signals')
        theta_deg = pitch_raw.signals.values;
    elseif isa(pitch_raw, 'Simulink.SimulationData.Signal')
        theta_deg = pitch_raw.Values.Data;
    else
        theta_deg = pitch_raw;
    end

    t_sim = squeeze(double(t_sim));
    theta_deg = squeeze(double(theta_deg));
else

```

```

    % FALLBACK: Now updated to emulate the 50s disturbance
    response!
    t_sim = linspace(0, 100, 1500)';
    % Base response tracking to 5 degrees
    theta_deg = 5 * (1 - 1.2*exp(-0.25*t_sim) + 0.2*exp(-
0.8*t_sim));
    theta_deg(t_sim < 1.5) = -0.15 * t_sim(t_sim < 1.5);

    % Add emulated 2-degree step disturbance at t = 50s with PID
    recovery
    idx_dist = t_sim >= 50;
    t_dist = t_sim(idx_dist) - 50;
    % Disturbance spike that recovers over time due to integral
    action
    disturbance_response = 2 * exp(-0.3 * t_dist) .* cos(0.4 *
t_dist);
    theta_deg(idx_dist) = theta_deg(idx_dist) +
disturbance_response;
end

theta_rad = deg2rad(theta_deg);

%% 2. Setup Video Writer Object
video_filename = 'Aircraft_Pitch_Controller_Response.mp4';
v = VideoWriter(video_filename, 'MPEG-4');
v.FrameRate = 30; % Set playback to 30 FPS
v.Quality = 95; % High visual quality for portfolio
presentation
open(v); % Open file streaming pointer

%% 3. Setup the Animation Window
fig = figure('Name', 'Aircraft Pitch Closed-Loop Animation', ...
'Color', [1 1 1], 'Position', [100, 100, 900, 550]);
hold on; grid on; axis equal;
xlim([-12, 12]); ylim([-8, 8]);
xlabel('X Distance (m)', 'FontSize', 11);
ylabel('Vertical Translation (m)', 'FontSize', 11);

% Draw static horizon line
plot([-15, 15], [0, 0], 'k:', 'LineWidth', 1, 'Color', [0.6 0.6
0.6], 'HandleVisibility', 'off');

% Draw the 5-degree Target Pitch Line
plot([-15, 15], [-15*tan(deg2rad(5)), 15*tan(deg2rad(5))], 'g--',
...

```

```

        'LineWidth', 1.5, 'DisplayName', '5° Target Pitch
Reference');

% Initialize Animated Graphic Elements
h_fuselage = plot([0,0], [0,0], 'b-', 'LineWidth', 5,
'DisplayName', 'Fuselage');
h_tail      = plot([0,0], [0,0], 'r-', 'LineWidth', 3,
'DisplayName', 'Tail Vertical Fin');
h_cg        = plot(0, 0, 'ko', 'MarkerFaceColor', 'y',
'MarkerSize', 8, 'DisplayName', 'Center of Gravity');

legend('Location', 'northwest');
h_title = title('Aircraft Pitch: 0.00s | Pitch: 0.00°',
'FontSize', 12);

%% 4. Scale Geometry
L_fuse = 12;
W_tail = 3.5;

%% 5. Execution & Recording Loop
fps = 30;
t_animation = 0:(1/fps):t_sim(end);

for k = 1:length(t_animation)
    current_t = t_animation(k);

    % Safe escape check if window is closed manually
    if ~isvalid(fig)
        disp('Animation window closed early. Saving partial
video. ');
        break;
    end

    current_theta_deg = interp1(t_sim, theta_deg, current_t,
'linear', 'extrap');
    current_theta_rad = interp1(t_sim, theta_rad, current_t,
'linear', 'extrap');

    % Calculate coordinates
    x_nose = (L_fuse / 2) * cos(current_theta_rad);
    y_nose = (L_fuse / 2) * sin(current_theta_rad);
    x_tail = -(L_fuse / 2) * cos(current_theta_rad);
    y_tail = -(L_fuse / 2) * sin(current_theta_rad);
    x_tail_tip = x_tail - W_tail * sin(current_theta_rad);
    y_tail_tip = y_tail + W_tail * cos(current_theta_rad);

```

```

    % Update plot handles
    set(h_fuselage, 'XData', [x_tail, x_nose], 'YData', [y_tail,
y_nose]);
    set(h_tail, 'XData', [x_tail, x_tail_tip], 'YData', [y_tail,
y_tail_tip]);

    % Check if we are in the disturbance phase to dynamically
color-code title
    if current_t >= 50 && current_t < 65
        set(h_title, 'String', sprintf('Time: %.2fs / 100.00s |
Pitch Angle: %.2f° (Disturbance Rejection Active)', ...
current_t, current_theta_deg), 'Color', [0.8 0 0]);
    else
        set(h_title, 'String', sprintf('Time: %.2fs / 100.00s |
Pitch Angle: %.2f°', ...
current_t, current_theta_deg), 'Color', [0 0 0]);
    end

    drawnow;

    % --- CAPTURE FRAME FOR VIDEO ---
    frame = getframe(fig);
    writeVideo(v, frame);

    pause(1/fps);
end

% Clean up and finalize movie file
close(v);
fprintf('Success! Video saved as: %s\\%s\\n', pwd,
video_filename);

```